程序设计

讲义

2026 年秋季

毕朝日

复旦大学集成电路与微纳电子创新学院

zhaori_bi@fudan.edu.cn https://bzr2915.github.io/bzr2915/

目录

第四讲　运算符与表达式 **4**
4.1　算出一个值，再把值存起来 4
4.1.1　运算符、操作数与表达式 4
4.1.2　赋值左边必须能指明可修改的对象 5
4.1.3　赋值表达式也有类型和值 6
4.2　除号怎样知道要不要保留小数 7
4.2.1　一元运算与二元运算 7
4.2.2　整数除法与浮点除法 8
4.2.3　把 256 秒拆成分钟和秒 8
4.3　表达式怎样组合，哪些动作有先后 10
4.3.1　优先级决定分组 10
4.3.2　分组规则不规定独立分支谁先求值 11
4.3.3　自增表达式取旧值还是新值 11
4.3.4　不能为没有规定顺序的冲突编造答案 13
4.3.5　复合赋值缩短的是写法 13
4.4　比较的结果也是一个整数 14
4.4.1　非零表示真，比较表达式产生 0 或 1 14
4.4.2　连续比较不是数学中的连写不等式 15
4.4.3　浮点数能比较，问题在于要比较什么 15
4.5　逗号什么时候真正规定先后 16
4.5.1　逗号表达式从左到右求值 16
4.5.2　分隔符不是逗号运算符 17
4.6　在一个整数里查看和修改二进制位 18
4.6.1　从十六进制 56 到八个二进制位 18
4.6.2　八位图不等于 C 只计算八位 19
4.6.3　置位、清位、翻转与检查 19
4.7　移位移动的是哪一种类型的位 20
4.7.1　无符号移位与位权 20
4.7.2　不要把有符号溢出画成无符号回绕 21

4.8 sizeof 为什么通常不执行括号里的运算 22
4.9 类型转换发生在计算的哪一步 22
4.9.1 整数提升不改变原对象的类型 22
4.9.2 共同运算类型不能只靠一条大小链判断 23
4.9.3 沿着一个字符追踪连续转换 24
4.9.4 转换不是一律“截掉高位” 25
4.9.5 显式转换写在哪里，结果就可能在哪里改变 25
4.10 表达式与语句的边界 26
4.10.1 有值不等于有作用，有作用也不一定有值 26
4.10.2 花括号把多项内容组成一个块 27
4.11 选读：把换算过程重复执行 27
4.11.1 while 先检查条件，再决定是否执行循环体 27
4.11.2 秒数输入：下一次输入也必须检查 29
4.11.3 条件本身也可能改变变量 30
4.11.4 一对花括号改变了哪些语句重复执行 31
4.12 选读：函数形参怎样决定转换 31
4.13 本讲运算符的优先级查阅表 33
4.14 习题与解析 33
习题一：每次转换发生在哪里 33
习题二：条件里的自增与循环体边界（选读） 34
习题三：商、余数与分钟数 36
习题四：表达式的值和对象的新值 37
习题五：补上真正需要的括号 37
习题六：逗号究竟属于哪一层 38
习题七：提取、改写与翻转一个四位字段 38
习题八：八位变量不等于八位运算 39
习题九：转换时机与有符号、无符号混合 39
习题十：检查一段换算与位检查程序 40
4.15 回到鞋码表的一行计算 41

第四讲　运算符与表达式

一家鞋店要在尺码旁边列出估算的脚长。假设采用一个教学用的换算关系：脚长等于鞋码乘 0.325，再加 7.64；脚长的单位是英寸，1 英寸等于 2.54 厘米。这不是通用于各国鞋码的标准公式，只用来观察程序怎样计算一张表。

第三讲已经能把鞋码和脚长排成两列，现在需要让程序算出第二列。设 `shoe` 保存鞋码，`foot` 保存脚长，计算写成：

```
foot = 0.325 * shoe + 7.64;
```

这一行包含两类动作。乘法与加法算出一个数；等号把这个数写入 `foot`。如果要接着计算下一个鞋码，还可以写 `shoe = shoe + 1.0;`。数学等式 $x = x + 1$ 不能成立，程序中的这一行却有明确的用途：读取旧值，算出新值，再改写原来的存储位置。

计算结果还取决于数据的类型。`7 / 4` 与 `7.0 / 4` 使用同一个除号，前者得到 1，后者得到 1.75。给结果变量换一个类型，并不能自动改变已经完成的除法。要理解这样的程序，需要同时看清表达式怎样组合、运算用什么类型，以及哪些对象会被改写。本讲沿用 C11 的语言规则。

4.1　算出一个值，再把值存起来

4.1.1　运算符、操作数与表达式

在 `0.325 * shoe` 中，星号 `*` 规定做乘法，称为**运算符**；0.325 与 `shoe` 是参加这次运算的**操作数**。运算符与操作数按语法组成**表达式**，程序对表达式进行**求值**，也就是执行规定的计算。单独的常量 `7.64` 和变量名 `shoe` 也可以构成表达式。

较大的表达式可以包含较小的**子表达式**。在 `0.325 * shoe + 7.64` 中，乘积是加法的左操作数，7.64 是加法的右操作数。这里不必先保存一个名为“乘积”的变量，加法可以直接使用乘法的结果。

下面先算两个鞋码，不引入重复执行的语法。宏名 `SCALE` 表示乘数，`ADJUST` 表示最后加上的调整量；宏的写法与第三讲相同。

```c
#include <stdio.h>
#define ADJUST 7.64
#define SCALE 0.325

int main(void) {
    double shoe = 3.0;
    double foot;

    foot = SCALE * shoe + ADJUST;
    printf("shoe=%.1f, foot=%.3f inches\n", shoe, foot);
    shoe = shoe + 1.0;
    foot = SCALE * shoe + ADJUST;
    printf("shoe=%.1f, foot=%.3f inches\n", shoe, foot);
    return 0;
}
```

在常见的二进制浮点环境中，输出为：

```
shoe=3.0, foot=8.615 inches
shoe=4.0, foot=8.940 inches
```

按十进制公式计算，第一行的脚长为 $0.325 \times 3 + 7.64 = 8.615$，第二行为 $0.325 \times 4 + 7.64 = 8.94$。输出中的三位小数由`%.3f` 规定。变量里的浮点近似值与屏幕上保留几位小数，仍是两个不同的问题。

4.1.2 赋值左边必须能指明可修改的对象

第二讲把变量与内存联系起来。C 把用于保存数据的一块存储区域称为**对象**。变量名通常用来指明一个对象，数组元素表达式也可以指明对象，例如 `name[0]` 指明字符数组的第一个元素。

能够指明对象的表达式称为**左值**。名称中的“左”来自赋值的典型写法，但不是说它只能写在左边：`shoe` 在 `shoe + 1.0` 中用于读取对象的值，在赋值左边则用于确定写入位置。赋值左边要求的是**可修改左值**。只读对象和数组整体不能用普通赋值改写，因此“是左值”还不够。

```
int bmw = 0;
int j = 2, k = 3;
char name[] = "Morgan";
const int limit = 20;

bmw = 220;
k = j + 3 * (k + j);
name[0] = 'J';
```

上面三次赋值都能找到可修改的对象。k 原来为 3，右边先得到 $2 + 3(3 + 2) = 17$，再把 17 存入 k。下面列出的则是**不合法写法**，不能作为正确程序编译：

```
220 = bmw;
(j + k) = 5;
limit = 21;
name = "Jordan";
```

常量 220 与和 j + k 没有提供可写的目标对象；limit 受 const 限定；name 是数组整体。修改首字母应使用 name[0]，不能给整个数组直接赋一个字符串。

“右值”常用来指运算中取用的值，例如常量 220、从 shoe 读出的值或加法的结果。初学时，判断赋值是否合法，应直接问左边能否确定一个可修改对象，而不必把所有表达式硬分成“只能放左边”或“只能放右边”。

4.1.3　赋值表达式也有类型和值

赋值除了改写对象，本身也产生一个值。假设 x 和 y 都是 int，下面的语句先让 y 得到 18，再让 x 得到 23：

```
x = 5 + (y = 3 * 6);
```

y = 3 * 6 这个表达式的值是赋值后 y 所保存的 18，因此外层计算 $5 + 18$。在本讲这些普通数值变量的例子中，赋值表达式的类型就是目标变量的类型。C 中的赋值表达式本身不是左值，不能再当成赋值目标。

目标类型还决定存入时的转换。下面的程序先更新 b，再用更新后的 b 计算 c：

```c
#include <stdio.h>

int main(void) {
    float a = 10.0f, b = 32.5f;
    short c = 12;

    b = a + b;
    c = a + b;
    printf("b=%f, c=%hd\n", b, c);
    return 0;
}
```

```
b=42.500000, c=52
```

第二次加法是 $10.0 + 42.5 = 52.5$。目标 `c` 只能保存整数，52.5 向 0 截去小数部分，得到可由 `short` 表示的 52。这是赋值引起的**隐式转换**，即程序没有写转换符号，语言规则仍要求转换。它不同于程序员写出 `(int)` 的显式转换。

连续赋值 `x = y = 1;` 按 `x = (y = 1)` 组合。右边给 `y` 赋值产生 1，外层再把 1 赋给 `x`。这里的“从右向左”描述赋值运算符的**结合性**，不是所有表达式都从右向左执行的命令。

4.2 除号怎样知道要不要保留小数

4.2.1 一元运算与二元运算

加法 `12 + 24`、减法 `33 - 22`、乘法 `3 * 4` 都有两个操作数，属于**二元运算**。`-15` 中的负号只作用于一个操作数 15，属于**一元运算**。因此`-(12 - 54)` 里面的减号先构成差 -42，外面的负号对这个差取相反数，结果为 42。

一元正号 `+x` 不改变数值的正负，但可能改变表达式使用的类型。例如，若 `ch` 是 `char`，且 `int` 能够表示全部 `char` 值，`+ch` 会先把读取的字符值转换为 `int`，结果类型也是 `int`；变量 `ch` 自身的类型不变。这种让较小整数类型的值使用 `int` 或 `unsigned int` 参加运算的转换称为**整数提升**，完整规则在后面的类型转换一节展开。

数学中的相邻书写不能替代 C 的乘号：$3x$ 应写成 `3 * x`。C 也没有乘方运算符。计算平方可以写 `x * x`；计算一般的实数幂可以使用 `<math.h>` 中的 `pow` 函数。例如 `pow(3.5, 2.2)` 表示 $3.5^{2.2}$，两个参数与返回结果的类型都是 `double`。`^` 在 C 中表示按位异或，不能拿来写平方。

4.2.2　整数除法与浮点除法

把 7 件物品平均装进 4 个箱子，每箱先装整件物品，可以分到 1 件，还剩 3 件。若计算每箱平均承担的重量，则可能需要 1.75。C 根据参加除法的操作数类型选择计算规则。

```c
#include <stdio.h>

int main(void) {
    double a = 7 / 4;
    double b = 7.0 / 4;

    printf("%d %d %d\n", 5 / 4, 6 / 3, 7 / 4);
    printf("%.2f %.2f\n", 7.0 / 4.0, 7.0 / 4);
    printf("a=%.2f, b=%.2f\n", a, b);
    printf("%d %d %d\n", -7 / 4, 7 / -4, -7 / -4);
    return 0;
}
```

```
1 2 1
1.75 1.75
a=1.00, b=1.75
-1 -1 1
```

两个整数相除时，商的小数部分被舍弃。C11 规定向 0 截断，所以 $-7/4$ 得到 -1，不是向更小的整数取整得到 -2。一方是 `double`、另一方是整数时，整数先转成 `double`，再做浮点除法。

`double a = 7 / 4;` 先用整数 7 和 4 得到整数 1，随后才把 1 转换为 1.0。**等号左边的类型不会倒过来修改右边的运算规则。**若 `total` 和 `count` 都是整数，而需要浮点商，可写 `(double)total / count`；只在除完以后写 `(double)(total / count)` 已经太晚。

除数不能为 0。整数还有一个容易遗漏的边界：若数学上的商超出结果类型范围，整数除法同样没有定义。例如在常见补码 `int` 中，`INT_MIN / -1` 的正数结果无法由 `int` 表示。不要用“整数运算只丢小数”来解释这类超范围情况。

4.2.3　把 256 秒拆成分钟和秒

余数运算符`%` 要求两个操作数都是整数。`256 / 60` 得到完整的分钟数 4，`256 % 60` 得到剩下的 16 秒。前者回答“有多少组 60”，后者回答“分组后剩多少”。

```c
#include <stdio.h>
#define SEC_PER_MIN 60

int main(void) {
    int sec;
    if (scanf("%d", &sec) != 1) {
        printf("Invalid input.\n");
        return 1;
    }
    if (sec < 0) {
        printf("Seconds must be nonnegative.\n");
        return 1;
    }
    int min = sec / SEC_PER_MIN;
    int left = sec % SEC_PER_MIN;
    printf("%d seconds = %d minutes, %d seconds\n",
           sec, min, left);
    return 0;
}
```

输入一个可由 `int` 表示的非负整数。本例用第二讲已经解释过的 `if` 检查输入：条件成立才执行花括号中的内容，`return 1;` 结束程序，避免继续使用无效数据。输入 256 后，程序输出如下；键盘输入的回显不属于这行输出。

```
256 seconds = 4 minutes, 16 seconds
```

输入 1024 得到 17 分 4 秒；输入 60 得到 1 分 0 秒；输入 0 得到 0 分 0 秒。检查 `scanf` 的返回值只说明完成了多少次赋值，并不证明整行没有多余字符。例如 `256abc` 仍可能先成功读出 256。

若除数 $b \neq 0$，且商可由结果类型表示，整数除法与求余满足

$$a = (a/b) \times b + a \,\%\, b.$$

因此求余的符号不能只凭“模运算一定非负”来判断。下表中的除法都向 0 截断。

表 4.1: 由整数商推出余数

a	b	a/b	$a\,\%\,b$	代回检查
5	3	1	2	$5 = 1 \times 3 + 2$
-5	3	-1	-2	$-5 = (-1) \times 3 - 2$
5	-3	-1	2	$5 = (-1) \times (-3) + 2$
-5	-3	1	-2	$-5 = 1 \times (-3) - 2$

非零余数与被除数同号，余数的绝对值小于除数的绝对值。`a % 0` 不合法；前面商不可表示的情形也不能改成求余来回避。`5.0 % 3.0` 违反操作数的类型要求，浮点余数需要另用数学库函数。

4.3　表达式怎样组合，哪些动作有先后

4.3.1　优先级决定分组

设 `n` 为 6，`SCALE` 为 2。下面两种写法只差一对括号，却表示不同的计算：

```
butter = 25.0 + 60.0 * n / SCALE;
butter = (25.0 + 60.0 * n) / SCALE;
```

乘除的**优先级**高于加法，表示乘除与操作数结合得更紧。第一行按 `25.0 + ((60.0 * n) / SCALE)` 分组，得到 $25 + 360/2 = 205$；第二行让 25 也参加最后的除法，得到 $(25 + 360)/2 = 192.5$。

可以把第一行右侧画成一棵**表达式树**。树的末端是操作数，每个运算节点使用其下面两个分支产生的结果。图只描述组合关系，不是一张要求机器从左到右执行的时间表。

```
          +
        /   \
     25.0    /
           /   \
          *   SCALE
        /   \
     60.0    n
```

`*` 和`/`优先级相同，结合性都是从左向右。因此 `5 * 6 / 7` 是 `(5 * 6) / 7`，用整数得到 4；它不是 `5 * (6 / 7)`，后者得到 0。括号改变的是表达式的分组。

4.3.2　分组规则不规定独立分支谁先求值

在 `6 * 12 + 5 * 20` 中，两项乘积都必须提供加法所需的数值，但 C 没有要求先算左边乘积还是先算右边乘积。两边都只读常量，所以这一自由不影响最终结果 172。

如果操作会改写变量，执行顺序就可能影响理解。对程序状态的改变称为**副作用**；赋值、自增和自减都有副作用，输入输出也会产生副作用。“副”不是“不重要”，而是说这些动作不只是算出一个表达式的值。

一条完整的表达式语句结束后，其中的副作用已经完成，后面的语句才能按顺序继续。因此下面的两个分号建立了清楚的先后：

```
i = i + 1;
j = i;
```

但在一次函数调用的多个参数之间，参数求值的先后通常没有规定。逗号若只是分隔参数，并不命令参数从左到右计算。括号也只能改变分组，不能凭空给相互独立的操作规定先后。

4.3.3　自增表达式取旧值还是新值

`++` 把操作对象增加 1，`--`把操作对象减少 1。前置写法 `++i` 产生增加后的值；后置写法 `i++` 产生增加前的值，同时也会把 `i` 增加 1。这里的区别是表达式提供什么值，不是后置自增可以无限推迟到以后某条语句。

```c
#include <stdio.h>

int main(void) {
    int i = 5;
    int j = i++;
    printf("i=%d, j=%d\n", i, j);
    i = 5;
    j = ++i;
    printf("i=%d, j=%d\n", i, j);

    int k, m;
    i = j = k = 2;
    m = i++ + j++ + k++;
    printf("%d %d %d %d\n", i, j, k, m);
    i = j = k = 2;
    m = i++ + ++j + ++k;
    printf("%d %d %d %d\n", i, j, k, m);
    return 0;
}
```

```
i=6, j=5
i=6, j=6
3 3 3 6
3 3 3 8
```

两个赋值过程可以按“表达式提供的值”和“对象最后的值”分别记录：

```
                initial i   value used by j   final i
j = i++;            5             5             6
j = ++i;            5             6             6
```

后两组加法分别作用于三个不同变量。第一组提供 $2+2+2$，第二组提供 $2+3+3$；各自语句结束后，三个变量都为 3。它们不需要假定三个自增从左到右执行。

类似地，设 `y = 2`、`n = 3`，则 `nextnum = (y + n++) * 6;` 使 `nextnum` 得到 30、`n` 变成 4。若改成 `++n`，参与加法的是 4，结果为 36。`x * y++` 按 `x * (y++)` 组合，因为后置自增比乘法结合得紧。

自增和自减要求可修改左值。`name[0]++` 可以对字符数组元素运算；`(x * y)++`、`3++`、`--5` 和 `++(i + j)` 都不合法。普通算术变量单独执行 `i++;` 或 `++i;`，在数值没有越界时效果相同。

4.3.4 不能为没有规定顺序的冲突编造答案

下面是**故意错误的片段**。变量均已初始化，仍不能据此预测一个 C11 保证的输出：

```
j = (--i) + (i--);
printf("%10d %10d\n", num, num * num++);
i = i++;
```

第一行在加法两边修改同一个 `i`，两次修改之间没有规定先后。第二行既修改 `num`，又在没有规定先后的位置读取 `num`。第三行的自增修改与外层赋值修改之间同样缺少所需的先后关系。这样的冲突属于**未定义行为**：语言不保证程序的行为，不能把不同编译器显示不同数字理解为“几个答案任选一个”。

`num * num++` 外面再套一层括号不会消除冲突。若意图是先打印旧值及其平方，然后增加变量，应在数值范围允许时写成：

```
printf("%10d %10d\n", num, num * num);
num++;
```

同一变量在表达式中出现多次并不自动构成错误。`i + i` 只有读取；`i = i + 1` 的赋值更新发生在所需的旧值计算之后。需要检查的是有没有彼此缺少先后关系的冲突读写，而不是简单地数变量出现了几次。

4.3.5 复合赋值缩短的是写法

`x += 2;` 表示把 `x` 增加 2。对普通变量，可以把它理解为 `x = x + 2;`。其余复合赋值按相同方式把运算与赋值组合起来。

表 4.2: C11 的十种复合赋值运算符

运算类别	写法
加、减、乘、除、求余	`+=` `-=` `*=` `/=` `%=`
左移、右移	`<<=` `>>=`
按位与、按位异或、按位或	`&=` `^=` `|=`

位运算在后文展开。`j %= 3;` 是把余数存回 `j`，`j ^= k;` 是把按位异或结果存回 `j`。复合赋值仍受原运算的类型与取值范围限制。

复合赋值还有一个不能用抄写两遍左边来代替的性质：确定左操作数所指对象的表达式只求值一次。例如，若 `int a[2] = {10, 20}; int i = 0;`，执行 `a[i++] += 3;` 后，`a[0]` 为 13，`i` 为 1。把它机械地展开成含有两个 `i++` 的赋值会改变程序，并可能引入未定义行为。

下面这行也不能靠“赋值从右向左结合”得出确定答案：

```
int a = 12;
a += a -= a * a;  /* wrong: unsequenced conflicting access */
```

它按 a += (a -= a * a) 分组，但外层为加法读取 a 与内层修改 a 之间没有规定先后。若意图是先减去平方，再把新值加到自身，应明确写成两条语句：

```
int a = 12;
a -= a * a;
a += a;
```

现在第一条语句得到 -132，第二条得到 -264。这个数值属于拆开后的程序，不能倒推成原来冲突表达式的标准答案。

4.4　比较的结果也是一个整数

4.4.1　非零表示真，比较表达式产生 0 或 1

鞋码表需要判断当前鞋码是否还在给定范围内。shoe < 18.5 把两个数作比较，条件成立时产生整数 1，不成立时产生整数 0。C 中的四个关系运算符是 <、<=、>、>=；两个相等性运算符是 ==、!=，分别判断相等和不相等。教学中经常把这六个统称为比较运算符。

= 是赋值，== 才是相等比较。x = 5 会修改 x，并产生值 5；x == 5 只比较 x 与 5，得到 0 或 1。

```
#include <stdio.h>

int main(void) {
    int x = 3, y = 5;
    printf("%d %d %d\n", x < y, x == y, x != y);
    x = y > 2;
    printf("x=%d, y=%d\n", x, y);
    return 0;
}
```

```
1 0 1
x=1, y=5
```

比较的优先级高于赋值，所以 x = y > 2 按 x = (y > 2) 组合。y 为 5，比较得到 1，再把 1 存进 x。算术运算又比比较结合得更紧，x > y + 2 表示 x > (y + 2)。

在 `if` 或 `while` 的条件位置，数值 0 表示假，非零表示真，负数也是真。因此数值 -3 能让条件成立；但不能反过来说比较运算符会产生 -3。**条件接受任意非零数表示真，比较表达式本身只产生 `int` 类型的 0 或 1。**

4.4.2 连续比较不是数学中的连写不等式

假设 `x` 为 100。数学上的 $0 < x < 10$ 不成立，但 C 表达式 `0 < x < 10` 按 `(0 < x) < 10` 组合。第一个比较得到 1，第二个比较变成 `1 < 10`，结果仍为 1。

要让两个条件同时成立，需要**逻辑与**运算符 `&&`：

```
(0 < x) && (x < 10)
```

`&&` 在两个操作数都为真时得到 1，否则得到 0。左边为假时，整个条件已经不可能成立，右边不再求值；左边为真时，才检查右边。这种跳过不必要计算的规则称为**短路求值**。

逻辑或 `||` 在两个操作数中至少一个为真时得到 1，两个都为假时得到 0。因此左边已经为真时，会跳过右边的求值。**逻辑非**`!` 把真变假、假变真：操作数为 0 时得到 1，非零时得到 0。这三个运算符的结果都是 `int` 类型的 0 或 1。

短路规则还可以保护右边的运算。例如，`d != 0 && n / d > 2` 先检查除数，`d` 为 0 时不会求商。这里仍需保证 `n / d` 在非零除数下没有超范围；短路只解决明确写在左边的检查问题。

四个关系运算符比 `==` 和`!=` 结合得更紧。相等性运算内部从左向右结合，因此 `ex != xyz == zee` 表示 `(ex != xyz) == zee`：先得到 0 或 1，再与 `zee` 比较。它不是“`ex` 不同于 `xyz`，而 `xyz` 等于 `zee`”的缩写。

4.4.3 浮点数能比较，问题在于要比较什么

六个比较运算符都可以用于浮点数，包括 `==`、`!=`、`<=` 和 `>=`。困难在于，十进制计算中应当相等的两个数学结果，经过二进制表示与舍入后，可能不是同一个浮点值。

下面在采用 IEEE 754 二进制 64 位 `double`、通常的舍入方式、未启用改变浮点语义的优化时，观察 0.1 与 0.2 相加。函数 `fabs` 来自 `<math.h>`，返回 `double` 参数的绝对值。

```c
#include <stdio.h>
#include <math.h>

int main(void) {
    double a = 0.1 + 0.2;
    double b = 0.3;
    double tolerance = 1e-12;
    printf("%.17g\n%.17g\n", a, b);
    printf("equal=%d, close=%d\n",
           a == b, fabs(a - b) <= tolerance);
    return 0;
}
```

```
0.30000000000000004
0.29999999999999999
equal=0, close=1
```

`a == b` 正确地报告存储的两个数不相同；`fabs(a - b) <= tolerance` 则回答它们之间的距离是否不超过给定容差。`1e-12` 表示 10^{-12}，这里只是为量级约为 1 的演示选取的容差，不能作为所有问题的固定标准。

若数据是以毫米计的测量值，容差也必须按毫米理解；若数值跨越很大量级，常需同时考虑绝对误差与相对误差。容差应来自问题允许的偏差。反过来，判断一个已知精确存储的 0.0 或一个只被赋予固定值的状态变量时，直接相等比较可以有明确用途。也不能断言 `3.0 * (1.0 / 3.0)` 必定不等于 1.0：两次舍入完全可能得到 1.0。

字符也能按编码值比较，例如 `ch != '$'`。但 `name1 == name2` 不能用来比较两段字符串的文字内容；字符数组在这种表达式中通常转换为指向首元素的指针，比较的是位置。第三讲的字符串知识说明，逐字内容是另一层问题，需要使用字符串比较函数，例如 `<string.h>` 中的 `strcmp`；其返回 0 才表示内容相同。

4.5　逗号什么时候真正规定先后

4.5.1　逗号表达式从左到右求值

前面说明，多数二元运算符不规定左右操作数谁先求值。**逗号运算符**是明确规定顺序的情形：先完成左操作数的求值与副作用，再求右操作数；整个表达式的类型和值取自右操作数。

例如，邮资计算中 `ounce` 记录以盎司为单位的重量，`cost` 记录费用，`NEXT_OZ` 表示

每增加一盎司的费用；在这些变量已经初始化、运算不越界的前提下，`ounce++, cost += NEXT_OZ;` 先增加重量计数，再增加费用。这个逗号表达式的值是更新后的费用。

设 `x`、`y` 和 `z` 均为 `int`，执行

```
x = (y = 4, (z = ++y + 2) + 5);
```

时，逗号左边先把 `y` 设为 4。右边的 `++y` 把 `y` 改成 5 并提供 5，`z` 得到 $5+2=7$，右边整个加法得到 $7+5=12$。最后 `x` 为 12、`y` 为 5、`z` 为 7。

逗号优先级低于赋值，这使下面两行含义不同。假设 `price` 是 `int`：

```
price = 245, 700;
price = (245, 700);
```

第一行按 `(price = 245), 700` 分组，`price` 得到 245，整个逗号表达式产生 700，但这个结果没有被保存。第二行把逗号表达式作为赋值右侧，`price` 得到 700。由于常量 245 或 700 在相应位置没有实际作用，编译器可能提示表达式结果未使用；这类警告与未定义行为不是一回事。

4.5.2 分隔符不是逗号运算符

`int x, y, z;` 中的逗号分隔三个声明项，`printf("%d %d", x, y)` 中的逗号分隔函数参数。它们都不是规定从左到右求值的逗号运算符。

```
#include <stdio.h>

int main(void) {
    int x, y, z;
    x = y = 1;
    z = x++, y++, ++y;
    printf("%d,%d,%d\n", x, y, z);
    x = y = 1;
    z = (x++, y++, ++y);
    printf("%d,%d,%d\n", x, y, z);
    return 0;
}
```

```
2,3,1
2,3,3
```

第一条含逗号的语句是 `(z = x++), y++, ++y`。先把 `x` 的旧值 1 存入 `z`，把 `x` 改

成 2；然后 y 从 1 变 2；最后 y 从 2 变 3。第二组用括号把整个逗号表达式交给赋值，因此 z 接收最后的 3。

这里两次修改 y 之间有逗号运算符规定的先后关系，所以不属于前面那种冲突。为了阅读方便，普通程序通常可以把它拆成几条语句；学习逗号的用途，是为了读懂其真实含义，而不是尽量把动作压缩到同一行。

4.6　在一个整数里查看和修改二进制位

4.6.1　从十六进制 56 到八个二进制位

第一讲用二进制位解释加法电路。一个整数也可以把不同位置的位分别用于记录状态，例如某位表示设备是否就绪，某位表示是否允许中断。**按位运算**逐位处理整数的二进制表示，不把整个整数只看成“真”或“假”。

下面采用 unsigned int 保存数值，先只画低 8 位；图中的 56、F0 和 0F 都是十六进制，C 源代码要写成 0x56u、0xF0u 和 0x0Fu。后缀 u 表示使用无符号整数常量。十六进制一位对应四个二进制位，0x56 即 $5 \times 16 + 6 = 86$。

& 是按位与：两位均为 1 才得到 1；| 是按位或：至少一位为 1 就得到 1；^ 是按位异或：两位不同才得到 1。~ 是一元按位取反，逐位交换 0 和 1。C 没有独立的按位同或运算符，可以把异或结果取反。

表 4.3: 单个二进制位的运算规则

a	b	取反 a	与	或	异或	同或
0	0	1	0	0	0	1
0	1	1	0	1	1	0
1	0	0	0	1	1	0
1	1	0	1	1	0	1

下图各列表示同一位的位置。把 0x56u 与 0xF0u 作按位与，高四位保留，低四位全部清零。

```
bit position     7 6 5 4   3 2 1 0
0x56             0 1 0 1   0 1 1 0
0xF0             1 1 1 1   0 0 0 0
                 -------   -------
0x56 & 0xF0      0 1 0 1   0 0 0 0   = 0x50
0x56 & 0x0F      0 0 0 0   0 1 1 0   = 0x06
```

用于保留、清除或选择某些位置的位模式称为**掩码**。`0x0Fu` 的低四位都是 1，其余位都是 0，所以与它相与可以提取低四位。

4.6.2 八位图不等于 C 只计算八位

下面把常用组合放进一个完整程序。`%02X` 以大写十六进制输出无符号整数，最小宽度为 2，不足时补 0。

```
#include <stdio.h>

int main(void) {
    unsigned int a = 0x56u;
    printf("AND: %02X %02X\n", a & 0xF0u, a & 0x0Fu);
    printf("OR : %02X %02X\n", a | 0xF0u, a | 0x0Fu);
    printf("XOR: %02X %02X\n", a ^ 0xF0u, a ^ 0x0Fu);
    printf("NOT: %02X\n", (~a) & 0xFFu);
    printf("XNOR: %02X %02X\n",
           (~(a ^ 0xF0u)) & 0xFFu,
           (~(a ^ 0x0Fu)) & 0xFFu);
    return 0;
}
```

```
AND: 50 06
OR : F6 5F
XOR: A6 59
NOT: A9
XNOR: 59 A6
```

为什么取反后要加 `& 0xFFu`？`unsigned int` 至少有 16 个数值位，通常是 32 位，`~` 会对整个类型的数值位取反。只画八位时看到 `56` 变成 `A9`，没有画出的高位也会从 0 变成 1。掩码 `0xFFu` 只保留低八位，才得到图中那个八位结果。`%02X` 的宽度是下限，不能代替掩码截掉高位。

把 `a` 改成 `unsigned char` 也不表示取反只做八位。常见环境中，小整数参加运算时先提升为 `int`，运算实际发生在提升后的类型上。图示的位宽、表达式运算的位宽、最后存储对象的位宽，应分别辨认。

4.6.3 置位、清位、翻转与检查

设一个状态字 `flags` 的第 2 位表示某功能是否启用。位的位置从 0 开始编号，第 0 位的位权为 1，第 2 位的位权为 4。以下是彼此连续的操作，初始 `flags` 为 `0x50u`：

```
unsigned int flags = 0x50u;
unsigned int mask = 0x04u;
flags |= mask;
flags &= ~mask;
flags ^= mask;
int enabled = (flags & mask) != 0u;
```

按位或把指定位置 1，flags 变为 0x54u；与反掩码相与把该位清 0，恢复 0x50u；异或翻转该位，再得到 0x54u。最后相与得到 4，和 0 比较得到 1，所以 enabled 为 1。

检查位时括号不能随意省略。flags & mask != 0u 会按 flags & (mask != 0u) 组合，因为!= 比按位与优先级高。这里 mask != 0u 变成 1，检查的反而是最低位。

按位与 & 与逻辑与 && 也不同：2u & 4u 得到 0，因为 010 和 100 没有共同为 1 的位置；2u && 4u 得到 1，因为两个操作数都非零。按位与不提供短路保护。

4.7　移位移动的是哪一种类型的位

4.7.1　无符号移位与位权

a << n 把 a 左移 n 位，a >> n 右移 n 位。两个操作数都须为整数，先分别进行整数提升，结果类型是提升后的左操作数类型。移位量必须非负，并且小于提升后左操作数类型的位宽。

对无符号整数，左移从低端补 0，高端移出的位丢弃；右移从高端补 0，低端移出的位丢弃。下面只画低八位，数值 64 的高位全为 0：

```
64u          0100 0000
64u >> 1     0010 0000    = 32
64u >> 3     0000 1000    =  8
64u << 1     1000 0000    = 128
64u << 2   1 0000 0000    = 256
```

最后一行需要九个位，unsigned int 能够容纳 256，所以结果不是 0。若明确只保留低八位，可以写 (64u << 2) & 0xFFu，这才得到 0。八位图不能代替 C 中的类型信息。

若提升后的无符号类型有 W 个数值位，左移相当于把 $a2^n$ 对 2^W 取模；在结果没有超出范围时，就是普通的乘 2^n。右移等于无符号整数除以 2^n 所得的商。因此 14u << 1、14u << 2 分别得到 28、56；14u >> 1、14u >> 2 分别得到 7、3。

移位还能把掩码定位到指定位置。1u << 2 得到 4，可以替代前节的 0x04u。从 0x56u 中取出高四位的数值，可以先右移 4 位，再与 0x0Fu 相与，得到 5。

4.7.2 不要把有符号溢出画成无符号回绕

对于非负有符号整数，左移要求数学上的结果仍可由结果类型表示；不满足时是未定义行为。对负有符号整数做左移也是未定义行为。C11 中负有符号数的右移结果由实现规定，很多实现采用补符号位的算术右移，但不能把这种做法当成所有 C 实现的保证。

在 32 位 `int` 中，`int m = 0x40000000;` 让 `m` 保存 1073741824。`m << 1` 的数学值为 2147483648，已经超过 `int` 的最大值 2147483647，所以不能把结果固定写成 −2147483648。`m << 2` 同样不能以“高位丢掉，所以为 0”解释。

若目的是观察位模式的回绕，应明确使用 32 位 `unsigned int`，并匹配输出格式：

```
#include <stdio.h>
#include <limits.h>

int main(void) {
    if (UINT_MAX != 0xFFFFFFFFu) {
        printf("This example requires 32 value bits.\n");
        return 1;
    }
    unsigned int m = 0x40000000u;
    unsigned int x = 0xFFFFFFFFu;
    printf("%#x %u\n", m << 1, m << 1);
    printf("%#x %u\n", m << 2, m << 2);
    printf("%#x %u\n", x >> 1, x >> 1);
    return 0;
}
```

```
0x80000000 2147483648
0 0
0x7fffffff 2147483647
```

`UINT_MAX` 来自 `<limits.h>`，给出 `unsigned int` 的最大值。程序先检查位宽条件；输出使用`%u` 解释无符号十进制值，`%#x` 显示带前缀的十六进制值，零值不加 `0x` 前缀。

`int n = 0xFFFFFFFF;` 也不是可移植地把 `n` 设为 −1。在常见 32 位环境中，该常量为 `unsigned int`，再转为无法表示该值的 `int`，结果由实现规定，或引发实现规定的信号。即使某实现得到 −1，随后 `n >> 1` 仍要查看该实现对负数右移的规定。

若实现对负数采用算术右移，`-64 >> 1` 会得到 −32，`-64 >> 3` 会得到 −8；无符号数 `192u >> 1` 与 `192u >> 3` 则分别得到 96 和 24。把低八位都画成 `1100 0000`，不

能据此说两种数相同：在常见 32 位补码表示中，−64 的高位为 1，192 的高位为 0，且它们的类型可能选择不同的右移规则。

移位量为负，或在 32 位提升类型上写 `1u << 32`，都属于未定义行为；处理器指令可能对移位量取低几位，但 C 程序不能依赖这种偶然结果。

4.8　sizeof 为什么通常不执行括号里的运算

第三讲用 `sizeof` 测量数组容量。它是运算符，结果类型为 `size_t`，这是实现选择的一种无符号整数类型，不保证总是 `unsigned int` 或 `unsigned long`。输出仍用`%zu`。

`sizeof(int)` 对类型求大小，括号必须写；`sizeof n` 与 `sizeof(n)` 对表达式求大小，都合法。对本讲使用的普通数值对象和固定长度数组，`sizeof` 只需要操作数的类型，不对操作数求值。

```
#include <stdio.h>

int main(void) {
    int n = 3;
    size_t bytes = sizeof(n++);
    printf("n=%d, bytes=%zu\n", n, bytes);
    return 0;
}
```

在 `sizeof(int) == 4` 的环境中，输出为：

```
n=3, bytes=4
```

`n++` 在这里没有执行，所以 `n` 没有增加；`sizeof` 看到其类型为 `int`，据此给出大小。C 还存在长度在运行时确定的变长数组，其相关 `sizeof` 规则可能涉及求值；本讲不使用这种数组，因此不把“永不求值”写成无条件结论。

4.9　类型转换发生在计算的哪一步

4.9.1　整数提升不改变原对象的类型

假设 `char ch = 'C';`，执行 `ch = ch + 1;` 时，是否真的只用一个字符大小的运算就完成了加法？在采用 ASCII 兼容字符集、`int` 能表示全部 `char` 值的环境中，`ch` 先从字符类型提升为 `int`，其值为 67，再与整数 1 相加，最后把 68 转换为 `char` 存回 `ch`，也就是字母 D。

这种把较小整数类型用于较大整数类型计算的规则称为**整数提升**。对 `_Bool`、`char`、

signed char、unsigned char、short、unsigned short 这些常用类型，如果 int 能够表示原类型的全部值，就转成 int；否则转成 unsigned int。例如 16 位 unsigned short 与 32 位 int 搭配时提升为 int，两者均为 16 位时则不能这样概括。

提升改变的是参加运算的值所使用的类型，原对象 ch 仍然是 char。整数提升用于许多整数运算、移位、一元运算及某些函数参数位置；不能说“小类型只要出现在任意表达式中就一定提升”。前面的 sizeof ch 仍测量 char 的大小，直接赋值的目标对象也不会因此变大。

4.9.2 共同运算类型不能只靠一条大小链判断

对本讲涉及的实数算术类型，加减乘除以及比较通常需要让两个操作数使用一个共同类型，这套规则称为**通常算术转换**。若含 long double，另一方转成 long double；否则若含 double，另一方转成 double；否则若含 float，另一方转成 float。这些规则没有说两个 float 相加就必定变成 double。

若两边都是整数，则先做整数提升，再处理类型差异。标准整数类型的转换等级依次为 signed char、short、int、long、long long，对应无符号类型具有相同等级。“等级”是语言规定的转换次序，不等于本机占多少字节；例如 long 与 int 可能同样宽，但等级不同。

表 4.4: 整数提升之后，怎样确定共同整数类型

两种类型的关系	转换办法
类型相同	无需再转换。
同为有符号，或同为无符号	转成等级较高的类型。
有符号与无符号混合，且无符号方等级不低	有符号方转成无符号方的类型。
有符号方等级较高，且能表示无符号方的全部值	无符号方转成该有符号类型。
有符号方等级较高，但不能表示无符号方的全部值	双方转成与该有符号类型对应的无符号类型。

例如 long 与 unsigned int 混合时，64 位 long 通常能容纳 32 位 unsigned int 的全部值，可以用 long；若两者都为 32 位，则通常转成 unsigned long。因此一条固定的“unsigned int 比 long 低”链不能说明所有实现。

-1 < 1u 是另一个典型例子。整数 -1 与 unsigned int 的 1 比较时，-1 先转成 unsigned int，得到该类型的最大值 UINT_MAX，所以比较结果为 0。原来的 -1 并没有神秘地变成正数；是比较规则选择了共同的无符号类型。

通常算术转换也不保证转换前后的数学值都精确保留。较大的整数转成 `float` 可能失去低位精度，正如第二讲中大浮点数加 1 可能不变。混合类型不是一律禁止，关键是知道哪一步转换成什么类型，以及范围和精度是否足够。

4.9.3　沿着一个字符追踪连续转换

下面采用 ASCII 兼容字符集，并假定 `int` 能表示全部 `char` 值。三个变量分别保存字符、整数和浮点数。

```
#include <stdio.h>

int main(void) {
    char ch;
    int i;
    float fl;

    fl = i = ch = 'C';
    printf("ch=%c, i=%d, fl=%.2f\n", ch, i, fl);
    ch = ch + 1;
    i = fl + 2 * ch;
    fl = 2.0 * ch + i;
    printf("ch=%c, i=%d, fl=%.2f\n", ch, i, fl);
    return 0;
}
```

```
ch=C, i=67, fl=67.00
ch=D, i=203, fl=339.00
```

第一次连续赋值让 `ch` 得到字符 C，表达式 `ch = 'C'` 的值随后转成 `int` 赋给 `i`，再转成 `float` 赋给 `fl`。下一次字符加法把 `ch` 更新为 D，对应 68。

在 `i = fl + 2 * ch;` 中，`ch` 提升为 `int`，乘法先得到整数 136。加法遇到 `float` 类型的 `fl`，把 136 转成 `float`，得到 `float` 类型的 203.0；赋给 `i` 时再转成整数 203。这里没有 `double` 操作数，不能因为出现浮点数，就把表达式类型写成 `double`。

随后 `2.0 * ch + i` 中的 `2.0` 是 `double` 常量，乘法、加法使用 `double`，得到 339.0 后再赋给 `float` 变量 `fl`。`printf` 接收 `fl` 时还有一次提升为 `double` 的过程，第三讲已经讨论过；这是函数参数位置的规则，不能搬来替代上面的加法规则。

```
ch = ch + 1
char 67 --> int 67 --> int 68 --> char 68

i = fl + 2 * ch
char 68 --> int 68 --> int 136 --> float 136.0
float 67.0 + float 136.0 --> float 203.0 --> int 203

fl = 2.0 * ch + i
double 2.0 * double 68.0 + double 203.0
                --> double 339.0 --> float 339.0
```

图中的箭头表示值在相应步骤转换类型，不表示 ch、i 或 fl 自身的声明类型发生变化。C 允许某些浮点中间值使用更高的计算精度，但这不改变表达式按语言规则得到的类型。

4.9.4 转换不是一律“截掉高位”

把浮点数转成整数时，先向 0 舍去小数部分，再要求所得整数能由目标类型表示。`int n = -3.8;` 得到 -3，不是 -4。如果整数部分超出目标范围，行为未定义，不能解释为只保留某几个字节。

例如在 8 位 char 环境中，下面的语句属于**错误示例**：

```
char ch;
ch = 5212205.17;
```

5212205 显然不能由 8 位 char 表示,因此没有可移植的字符输出答案。加上 (char) 只是明确写出同一次转换，不能让超范围转换变得合法。

整数转成无符号整数则有不同规则：不断加减目标范围大小，得到落在目标范围内的唯一值。例如 8 位 unsigned char 的范围为 0 至 255，(unsigned char)300 得到 44；(unsigned char)-1 得到 255。整数转成不能表示其值的有符号类型，C11 规定结果由实现规定，或引发实现规定的信号。这与浮点转整数超范围的未定义行为又不同。

因此，检查类型转换至少要说明源类型、目标类型、数值范围。编译器给出“可能丢失数据”的警告时，应该核对这些条件，而不是只加一个转换符号使警告消失。

4.9.5 显式转换写在哪里，结果就可能在哪里改变

显式转换的写法为（类型名）表达式，也常叫强制类型转换。它产生指定类型的值，并不改变原变量的声明。下面三次赋值分别在不同阶段舍去小数部分：

```c
#include <stdio.h>

int main(void) {
    int mice;
    mice = 1.6 + 1.7;
    printf("%d\n", mice);
    mice = (int)1.6 + (int)1.7;
    printf("%d\n", mice);
    mice = (int)(1.6 + 1.7);
    printf("%d\n", mice);
    int total = 7, count = 4;
    printf("%.2f %.2f\n", (double)total / count,
           (double)(total / count));
    return 0;
}
```

```
3
2
3
1.75 1.00
```

第一种与第三种先做浮点加法，得到约 3.3，再截断为 3；第二种先把两个操作数都截断为 1，再用整数加法得到 2。最后一行再次说明，转换必须发生在需要改变的运算之前。

对较小整数类型运算时也要注意存储时刻。假设 8 位 `unsigned char c = 250;` 且 `int` 为 32 位，`c + 10` 的类型是 `int`、值为 260；若执行 `c += 10;`，加法结果再转换回 `unsigned char`，`c` 得到 4。回绕发生在无符号转换时，不是因为这次加法本来只用了八位。

4.10　表达式与语句的边界

4.10.1　有值不等于有作用，有作用也不一定有值

`dogs = 3` 是赋值表达式；加上分号写成 `dogs = 3;`，就成为一条**表达式语句**。赋值表达式的值 3 在此没有继续使用，但给 `dogs` 写入 3 的副作用仍然保留。

`3 + 4;` 也是合法的表达式语句，只是算出的 7 被丢弃，也没有可观察的副作用；编

译器可能直接去掉这次计算。printf("Hello\n"); 则调用函数输出文字，返回的字符数虽未保存，输出已经发生。

并不是每个表达式都有可供使用的值。返回类型为 void 的函数调用可以产生输出，却不提供结果数值；void 在这里表示没有返回值。后面的选读函数 pound 就是这种情况，不能写 int n = pound(5); 来接收不存在的值。

只含一个分号的; 是**空语句**，不执行操作。int count, sum; 是声明，说明两个对象的类型；严格按 C 语法，它不是表达式语句，去掉分号也不会得到一个表达式。return 0; 是返回语句，结束当前函数。if、while 各自有专门的语法，不能用“所有语句都是表达式后加分号”来定义语句。

4.10.2 花括号把多项内容组成一个块

用 {} 包围的声明和语句组成**复合语句**，也称**块**。前面的输入检查中，if 后面的块把错误提示与 return 1; 组成同一个受条件控制的整体。

缩进帮助人阅读，花括号才规定多条语句属于哪个块。例如，不能通过把一条语句向右多缩进几个空格，就让它自动归属于前面的 if 或 while。控制语句后是否写分号同样会改变意义；多写一个空语句不总是无害。

4.11 选读：把换算过程重复执行

选读范围。本节为了读懂重复打印的完整例子，介绍 while 的最小语法。前面各节的运算规则不依赖本节；涉及循环的习题在读完本节后再做。

4.11.1 while 先检查条件，再决定是否执行循环体

while（条件）语句称为 while 循环。每次先求条件值；为 0 则结束循环，非零则执行后面的语句，执行完再回去检查条件。while 后受它控制的那条语句称为**循环体**；需要重复执行多条语句时，用花括号把它们组成一个块，让整个块充当循环体。

鞋码从 3.0 开始，每次增加 1.0，只要小于 18.5 就打印一行：

```
#include <stdio.h>
#define ADJUST 7.64
#define SCALE 0.325

int main(void) {
    double shoe = 3.0;
    printf("Shoe size(men's)    foot length\n");
    while (shoe < 18.5) {
        double foot = SCALE * shoe + ADJUST;
        printf("%10.1f %15.2f inches\n", shoe, foot);
        shoe = shoe + 1.0;
    }
    printf("If the shoe fits, wear it.\n");
    return 0;
}
```

条件第一次读取 3.0，判断为真，打印鞋码 3.0 对应的脚长。循环体末尾把鞋码改成 4.0，再次检查。鞋码 18.0 仍小于 18.5，打印后增加到 19.0；19.0 不满足条件，因此不打印这一行。总共打印 16 行数据。

按十进制公式，三个有代表性的状态如下。脚长的屏幕显示保留两位小数，正好靠近舍入边界的数可能受二进制近似影响；行数与鞋码边界不依赖这种小数显示差异。

表 4.5: 鞋码循环中的条件和状态

检查时 `shoe`	条件	十进制公式脚长	循环体之后
3.0	真	8.615	`shoe = 4.0`
18.0	真	13.49	`shoe = 19.0`
19.0	假	不计算	离开循环

把循环体末尾的更新替换成 `++shoe;`，本例仍然打印同一组鞋码。若把自增放进条件，则初值也必须配套调整：

```
double shoe = 2.0;
while (++shoe < 18.5) {
    double foot = SCALE * shoe + ADJUST;
    printf("%10.1f %15.2f inches\n", shoe, foot);
}
```

第一次检查时先从 2.0 增加到 3.0，才比较 18.5，因此仍从 3.0 开始打印。若保留初值 3.0，就会从 4.0 开始，漏掉第一行。浮点变量也能自增，本例的这些小整数在常见浮点格式中均能精确表示。

4.11.2　秒数输入：下一次输入也必须检查

只输入一次秒数的程序可以改成重复换算，约定输入 0 或负数结束。循环内部的再次输入仍可能失败，必须检查，不能沿用旧秒数无限重复。

```
#include <stdio.h>
#define SEC_PER_MIN 60

int main(void) {
    int sec;
    printf("Enter seconds (<=0 to quit):\n");
    if (scanf("%d", &sec) != 1) {
        printf("Invalid input.\n");
        return 1;
    }
    while (sec > 0) {
        int min = sec / SEC_PER_MIN;
        int left = sec % SEC_PER_MIN;
        printf("It is %d minutes, %d seconds.\n", min, left);
        printf("Enter next value (<=0 to quit):\n");
        if (scanf("%d", &sec) != 1) {
            printf("Invalid input.\n");
            return 1;
        }
    }
    printf("Done!\n");
    return 0;
}
```

依次输入 256、1024、-1，两次换算分别得到 4 分 16 秒与 17 分 4 秒，然后输出 Done!。本例的输入约定仍是可由 int 表示的整数；scanf 返回值不负责证明任意长数字都在范围内。

4.11.3　条件本身也可能改变变量

while (n) 把非零的 n 视为真。若 n 从 3 逐次减 1，会打印 3、2、1，再因 0 退出；若从 −3 逐次加 1，会打印 −3、−2、−1，同样因 0 退出。

```
#include <stdio.h>

int main(void) {
    int n = 3;
    while (n) {
        printf("%2d is true.\n", n--);
    }
    printf("%2d is false.\n", n);
    n = -3;
    while (n) {
        printf("%2d is true.\n", n++);
    }
    printf("%2d is false.\n", n);
    return 0;
}
```

参数 n--把旧值提供给 printf，同时修改 n。在进入函数执行之前，这次参数求值中的修改已经完成；下一次循环检查因而读取新值。这里没有其他参数同时读取 n，不存在前面那个 printf 错例中的冲突。

再看求 $1+2+\cdots+20$ 的程序：

```
#include <stdio.h>

int main(void) {
    int count = 0, sum = 0;
    while (count++ < 20) {
        sum = sum + count;
    }
    printf("sum=%d, count=%d\n", sum, count);
    return 0;
}
```

```
sum=210, count=21
```

第一次条件比较使用旧值 0，比较成立；到循环体开始时 count 已经是 1，因此先

累加 1。旧值 19 小于 20 时，count 变成 20，累加 20。最后一次检查使用旧值 20，比较失败，但 count++ 仍执行，于是离开循环时 count 为 21。条件判断失败不意味着条件表达式的副作用被撤销。

4.11.4　一对花括号改变了哪些语句重复执行

下面两个片段中，i、s 均为 int。第一段只重复赋值，结束后打印一次；第二段每轮赋值后立即打印。

```
i = 0;
while (i++ < 10)
    s = 10 * i + 5;
printf("s=%d\n", s);

i = 0;
while (i++ < 10) {
    s = 10 * i + 5;
    printf("s=%d\n", s);
}
```

第一段输出 s=105 一次；第二段依次输出 s=15、s=25，直到 s=105，共十行。两段退出时 i 都是 11，原因仍是最后一次失败检查也执行了后置自增。

若在 while (i < 10) 后误写分号，循环体会变成空语句。只要条件一直成立，而空语句又不修改 i，后面的代码就无法按原意推进。读循环时应实际找出受控制的那一条语句或那个块。

本节选读结束。

4.12　选读：函数形参怎样决定转换

选读范围。本节只用一个打印符号的函数说明参数转换。函数设计与其他控制结构不作为前文的学习前提。

形参是函数定义中用来接收输入的对象，**实参**是调用时提供的表达式。void pound(int n); 是函数原型，告诉编译器 pound 接收一个 int 参数、不返回值；声明末尾的分号说明这里还不是函数体。

下面的函数用字符 # 画一行。调用 pound(5) 时，形参 n 得到数值 5 的副本；改变 n 不改变调用者的原变量。

```c
#include <stdio.h>
void pound(int n);

int main(void) {
    int times = 5;
    char ch = '!';
    float f = 6.0f;
    pound(times);
    pound(ch);
    pound((int)f);
    return 0;
}

void pound(int n) {
    while (n-- > 0) {
        printf("#");
    }
    printf("\n");
}
```

在 ASCII 兼容字符集中，`'!'` 的编码值是 33，因此三行分别有 5、33 和 6 个 `#`：

```
#####
#################################
######
```

`pound(ch)` 把字符值转换为形参要求的 `int`。`pound((int)f)` 显式把 6.0 转成 6；因为原型已经规定参数类型为 `int`，这里写 `pound(f)` 也会按参数要求隐式转换。本例中的 6.0 在范围内，因此两种写法效果相同。

这与 `printf` 的可变参数不同。对于原型中有确定类型的参数，按对应形参类型转换；对于 `printf` 格式串后面的可变参数，`float` 提升为 `double`，较小整数类型进行整数提升。不能把所有函数调用都描述为“`float` 必定转成 `double`”。

若把 `f` 改为 6.9，按 `int` 参数传递会得到 6；若 `f` 的整数部分超出 `int` 范围，隐式与显式两种转换都不能保证合法。函数原型帮助检查接口，不会自动使任意数值适合目标类型。这里的 `pound` 只演示给定的小非负计数，不作为处理任意整数输入的通用函数。

本节选读结束。

4.13　本讲运算符的优先级查阅表

下表按优先级由高到低列出本讲实际用到的运算符，同一行中的运算符优先级相同。这是本讲范围的查阅表，不是 C 语言所有运算符的完整列表；例如成员访问、条件运算符没有在本讲展开。用于分组的括号直接指定子表达式，不另作为普通二元运算符列入。

表 4.6: 本讲所用运算符的结合关系

运算符或写法	结合方向
函数调用 `f(...)`、下标 `a[i]`、后置 `++ --`	从左向右
前置 `++ --`、一元 `+ -`、`! ~`、取地址 `&`、`sizeof`	从右向左
显式类型转换 `(类型名)`	从右向左
`* / %`	从左向右
二元 `+ -`	从左向右
`<< >>`	从左向右
`< <= > >=`	从左向右
`== !=`	从左向右
按位与 `&`	从左向右
按位异或 `^`	从左向右
按位或 `|`	从左向右
逻辑与 `&&`	从左向右
逻辑或 `||`	从左向右
`=` 及十种复合赋值	从右向左
逗号运算符,	从左向右

第二、三讲中 `scanf` 的 `&n` 是取地址，这里的 `a & b` 是按位与，操作数个数与上下文决定含义。表中的结合方向用于补出分组括号；真正保证求值先后的逗号、逻辑与、逻辑或，应按各自规则理解，不能从其他行的“从左向右”推断出相同保证。

4.14　习题与解析

习题一：每次转换发生在哪里

所有变量都是 `int`。下面四条语句分别独立执行，求语句结束后被赋值变量的值，并说明中间表达式的类型。

```
y = x = (2 + 3) / 4;
y = 3 + 2 * (x = 7 / 2);
x = (int)3.8 + 3.3;
x = 22.0 * 3 / 5;
```

解析。第一条先做整数加法得到 5，再做整数除法 5/4 得到 1，连续赋值使 `x` 与 `y` 都为 1。小括号改变了分组，没有把整数除法变成浮点除法。

第二条的 `7 / 2` 为整数 3，赋给 `x` 后，赋值表达式提供 3。因此 `y` 得到 $3+2\times3=9$，`x` 为 3。

第三条先把 3.8 显式转成整数 3，再与 `double` 类型的 3.3 相加。加法前 3 又转成 `double`，得到约 6.3，最后赋给 `x` 时向 0 截断，`x` 为 6。`(int)` 只作用于紧邻的 3.8，不包住后面的整个加法。

第四条由于 22.0 为 `double`，乘法得到 66.0，除法得到约 13.2，再转成 `int`，`x` 为 13。

习题二：条件里的自增与循环体边界（选读）

以下四组片段独立执行。预测输出，保留空格与换行，并说明能否正常到达片段末尾。第三组采用 ASCII 兼容字符集。

```
/* A */
int x = 0;
while (++x < 3)
    printf("%4d", x);

/* B */
int x = 100;
while (x++ < 103)
    printf("%4d\n", x);
printf("%4d\n", x);
```

后两组分别处理字符递增与字符串精度：

```
/* C */
char ch = 's';
while (ch < 'w') {
    printf("%c", ch);
    ch++;
}
printf("%c\n", ch);

/* D */
#define MESG "Hello! The World!"
int n = 1;
while (n < 5)
    printf("%.*s\n", n, MESG);
n++;
printf("END\n");
```

解析。 A 先把 x 从 0 增到 1，打印 1；再增到 2，打印 2；再增到 3，条件失败。每个字段宽度为 4，所以实际输出为下面这一行，每个数字前有三个空格，末尾没有换行。

```
   1   2
```

片段结束时 x 为 3。这里代码框单独占一行，不表示原程序打印了换行字符。

B 用旧值 100、101、102 分别与 103 比较，条件均成立；到循环体时 x 已分别增加到 101、102、103。最后旧值 103 使比较失败，但 x 仍增到 104，循环后的 printf 再打印 104：

```
101
102
103
104
```

C 循环内打印 s、t、u、v；ch 增加为 w 后不再进入循环，循环外打印 w 及换行，合起来是一行 stuvw。字母编码连续递增的推导依赖本题声明的 ASCII 环境。

D 的循环体只有 printf 一条语句，n++ 在循环外。n 始终为 1，条件一直成立，%.*s 每次只打印字符串的第一个字节，因此不断输出一行 H，无法正常到达 END。修正应把更新与输出放在同一个块内：

```c
#include <stdio.h>
#define MESG "Hello! The World!"

int main(void) {
    int n = 1;
    while (n < 5) {
        printf("%.*s\n", n, MESG);
        n++;
    }
    printf("END\n");
    return 0;
}
```

```
H
He
Hel
Hell
END
```

输出 `Hell` 后 `n` 变成 5，下一次条件失败。这道题的错误在循环体边界，而不在字符串精度规则。

习题三：商、余数与分钟数

求`-17 / 5`、`-17 % 5`、`17 / -5`、`17 % -5`。再把 3661 秒拆成小时、分钟和秒，要求余下的分钟和秒均小于 60。

解析。两个商都是 -3。利用 $a = (a/b)b + a\,\%\,b$，前一组有 $-17 = (-3)\times 5 - 2$，所以余数为 -2；后一组有 $17 = (-3)\times(-5) + 2$，所以余数为 2。

3661 除以 3600 得到完整小时数 1；余下 61 秒，再除以 60 得到 1 分钟、求余得到 1 秒。可以写成：

```c
int total = 3661;
int hours = total / 3600;
int rest = total % 3600;
int minutes = rest / 60;
int seconds = rest % 60;
```

`total / 60` 得到的是总分钟数 61，而不是去掉整小时后的分钟数。变量名 `rest` 明确保存哪一步留下的量，可以减少这种含义混淆。

习题四：表达式的值和对象的新值

下面三组分别独立执行，变量均为 int。求最后的值；若没有 C11 规定的确定行为，应指出原因。

```
/* A */
int i = 5;
int a = i++;
int b = ++i;

/* B */
int y = 2, n = 3;
int nextnum = (y + n++) * 6;

/* C: deliberately wrong */
int i = 5;
int j = i++ + ++i;
```

解析。 A 的第二行让 a 取得旧值 5，i 增加为 6；第三行再把 i 增为 7，b 取得新值 7。最后 i=7、a=5、b=7。

B 的乘法使用 $2+3$，所以 nextnum=30；语句结束后 n=4，y 仍为 2。

C 在加法的两个操作数中修改同一对象 i，没有规定两次修改的先后，属于未定义行为。若希望分别取一次后置自增的值和下一次前置自增的值，再相加，应拆成：

```
int i = 5;
int left = i++;
int right = ++i;
int j = left + right;
```

这段修正代码明确得到 left=5、right=7、j=12、i=7；这个答案不适用于原来的错误表达式。

习题五：补上真正需要的括号

设 x=100、flags=4u、mask=4u，求下列表达式的值并修正与意图不符的写法：

```
0 < x < 10
flags & mask != 0u
(flags & mask) != 0u
2u & 4u
2u && 4u
```

解析。第一式按 `(0 < x) < 10` 分组，结果为 1；若意图检查开区间 $(0, 10)$，应写 `0 < x && x < 10`，结果为 0。

第二式按 `flags & (mask != 0u)` 分组，变成 `4u & 1u`，得到 0；第三式先按位与得到 4，再与 0 比较，得到 1，因此第三式才是检查掩码位置的正确写法。最后两式分别得到无符号整数 0 和 `int` 类型的 1，区别来自按位运算与逻辑运算处理对象的方式。

变式：乘以 0 能代替短路吗？设 `int n = 7, d = 0;`，只作静态分析，不运行第二个错误表达式：

```
d != 0 && n / d > 2
(d != 0) * (n / d > 2)  /* wrong when d == 0 */
```

第一式左边为 0，逻辑与跳过右边，因此整个表达式为 0，不执行除法。第二式的乘法没有短路规则；虽然左操作数为 0，右边的除零仍使行为未定义，不能把结果算成 0。用 0 或 1 选择已经合法算出的值，与阻止某个表达式求值，是不同的要求。

习题六：逗号究竟属于哪一层

设 `x`、`y`、`z` 为 `int`。分别执行下面两组，求结果：

```
/* A */
x = y = 1;
z = x++, y++, ++y;

/* B */
x = (y = 4, (z = ++y + 2) + 5);
```

解析。 A 中赋值比逗号结合得更紧，实际结构为 `(z = x++)`, `y++`, `++y`。最终 `x=2`、`y=3`、`z=1`。把右侧全加括号会改变 `z` 的值，不能称为仅仅改善排版。

B 的逗号在赋值右侧括号内，先把 `y` 设为 4，再求 `(z = ++y + 2) + 5`，最终 `x=12`、`y=5`、`z=7`。如果把 `printf("%d %d", y++, ++y)` 当成同类例子，就把参数分隔符误当成逗号运算符了；这个调用的两次修改没有相应的先后保证。

变式。将 B 中的 `++y` 改为 `y++`，其余不变，三个变量的最终值是什么？逗号左边先令 `y=4`，后置自增向加法提供旧值 4，因此 `z=6`、`x=11`；语句结束时自增也已完成，`y=5`。`y` 的最终值虽然与原题相同，参与加法的值却不同。

习题七：提取、改写与翻转一个四位字段

`unsigned int value = 0x56u;`。提取高四位与低四位；保留高四位，把低四位改成 9；再翻转低四位。每一步都说明掩码的作用。

解析。高四位先右移到最低位置，再保留四位；低四位直接与 `0x0Fu` 相与：

```
unsigned int high = (value >> 4) & 0x0Fu;
unsigned int low = value & 0x0Fu;
value = (value & ~0x0Fu) | 0x09u;
value ^= 0x0Fu;
```

high 为 5、low 为 6。value & 0x0Fu 把低四位清零，并保留其他位，本题得到 0x50u；再与 0x09u 作按位或，得到 0x59u。最后与 0x0Fu 异或，低四位的 1001 翻转成 0110，因此恢复 0x56u。若先不清零而直接与 9 作按位或，低四位会变为 15，不能完成“改成 9”的要求。

习题八：八位变量不等于八位运算

采用 8 位 unsigned char 与 32 位 int。设 unsigned char c = 64;，说明 c << 2 的类型和值；若把结果再赋给 c 会怎样？32 位 int m = 0x40000000; 的 m << 1 能否按相同方式分析为合法回绕？

解析。 c 先提升为 int，c << 2 的结果为 int 类型的 256。256 能由 32 位 int 表示，因此移位合法。再执行 c = c << 2; 时，256 转换为 8 位无符号类型，对 256 取模得到 0。变化次序为：

```
unsigned char 64 --> int 64 --> int 256
                                    |
                                    v
                              unsigned char 0
```

m << 1 则在有符号移位这一步就超出了 int 范围，属于未定义行为，没有合法结果等待后续转换。若要回绕，必须在移位前选择合适的无符号类型，例如使用 unsigned int m = 0x40000000u;。

习题九：转换时机与有符号、无符号混合

解释下面各项。涉及 unsigned char 时采用 8 位字节，int 采用 32 位。

```
(double)(7 / 4)
(double)7 / 4
(int)1.6 + (int)1.7
(int)(1.6 + 1.7)
-1 < 1u
(unsigned char)300
(char)5212205.17
```

解析。前四项依次为 1.0、1.75、2、3。共同原因是转换只能影响转换之后的运算，无法恢复已经舍去的小数。

第五项为 0：比较前 -1 转成 `unsigned int` 的最大值。第六项为 44，属于整数转无符号整数的模转换。第七项从浮点数直接转成不能表示其整数部分的 `char`，行为未定义；不能把第六项的模规则套到第七项。

习题十：检查一段换算与位检查程序

下面程序意图读入两个小整数，输出浮点商，再检查 `flags` 的第 2 位。指出错误并给出可运行的修正版本。

```
/* Deliberately wrong. */
#include <stdio.h>
int main(void) {
    int total, count;
    unsigned int flags = 4u;
    scanf("%d%d", &total, &count);
    double average = total / count;
    printf("%.2f\n", average);
    printf("%d\n", flags & 4u != 0u);
    return 0;
}
```

解析。输入可能失败，两个变量不一定都被赋值；除数可能为 0；`total / count` 仍执行整数除法；位检查缺少分组括号，比较会先把 4 变成 1。位检查结果应先通过`!=` 得到 `int` 类型，再用`%d` 输出。

```
#include <stdio.h>

int main(void) {
    int total, count;
    unsigned int flags = 4u;
    if (scanf("%d%d", &total, &count) != 2) {
        printf("Invalid input.\n");
        return 1;
    }
    if (count == 0) {
        printf("Zero divisor.\n");
        return 1;
    }
    double average = (double)total / count;
    printf("%.2f\n", average);
    printf("%d\n", (flags & 4u) != 0u);
    return 0;
}
```

输入 `7 4`，输出两行 `1.75` 和 `1`；输入 `7 0`，提示零除数并退出；输入 `7 x`，提示输入无效并退出。题目约定输入小整数，这个检查没有把任意文本都变成合法数据，但已经在每一步计算前验证了本例需要的条件。

4.15　回到鞋码表的一行计算

`foot = SCALE * shoe + ADJUST;` 中的乘法和加法组成一个数值表达式，宏展开后操作数的类型决定采用浮点运算；赋值把结果转换成 `foot` 的类型并写入对象。`shoe = shoe + 1.0;` 读取旧鞋码再保存新鞋码，若改写成自增，仍要分清使用的是更新前还是更新后的值。

要从一行算式走到可解释的程序，可以沿着同一个过程检查：先按优先级与结合性确定分组，再确定各步使用的类型，接着找出读取和修改的对象，最后检查运算范围与确有保证的先后关系。秒数换算中丢失的小数、掩码中保留的位、循环最后一次失败判断留下的新值，都能由这些具体步骤推出。